



RAFFLES INSTITUTION

2024 YEAR 6 TERM 3 TIMED PRACTICE

CANDIDATE
NAME

CLASS

24

COMPUTING

9569

Practical

2 hours

Additional materials: Electronic version of TASK1_commands.TXT file
Electronic version of product.TXT data file
Electronic version of supplier.TXT data file
Electronic version of transaction.TXT data file
Electronic version of TreasureHuntServer.py file

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to 4 marks out of 70 will be awarded for the use of common coding standards for programming style.

The number of marks is given in brackets [] at the end of each task.

The total number of marks for this paper is 70.

FOR EXAMINER'S USE			
Task 1	Task 2	Task 3	
			TOTAL
			70

This document consists of **8** printed pages.

RAFFLES INSTITUTION
Mathematics Department

Instructions to candidates:

Your program code and output for each of Task 1 to Task 2.3, and Task 3 should be saved in a single `.ipynb` file using Jupyter Notebook. Task 2.4 should be saved in a single `.py` file.

Start each task by labelling with a `#`. For example, `#Task 1.1`.

Make sure that each of your `.ipynb` files shows the required output in Jupyter Notebook.

- 1 Name your Jupyter Notebook as `TASK1_<your class>_<your name>.ipynb`.

The company "ABC Enterprises" requires a program to manage information about its employees using a linked list. Each employee has a unique employee ID, a name, and a salary. Employee ID may consists of letters and numbers.

The employee records are sorted by their employee IDs for efficient access and management.

Task 1.1

Implement a Python class called `EmployeeNode` to represent an individual employee's information. Each node should contain the employee ID, name and salary. [3]

Task 1.2

In a new cell, implement a Python class named `EmployeeLinkedList` to manage the storage and manipulation of employee information within the linked list. The class will handle operations such as adding, removing, updating, and displaying employee details using these functions: [21]

Function	Description
<code>AddEmployee</code>	• Enable the user to add a new employee to the linked list by providing their employee ID, name, and salary. • Function can be called with employee details as parameters if they are available. Alternatively, if the parameters are not provided, the function prompts the user for input to obtain the employee details interactively. • Ensure that the new employee is inserted into the linked list in ascending order of their employee ID.
<code>RemoveEmployee</code>	• Allow the user to remove an employee from the linked list based on their employee ID. • Function should be called with employee ID as parameter.

DisplayAll	• Display the details of all employees (including their employee ID, name, and salary) currently stored in the linked list, in the order in which they are stored. • Use a recursive approach to traverse the linked list and display employee details.
UpdateSalary	• Allow the user to update the salary of an employee based on their employee ID. • Function can be called with employee ID and salary as parameters.

Task 1.3

In a new cell, copy and paste the code from the text file 'Task1_commands.TXT'.

By adding to your program code, perform the following tasks:

1. Add this employee into your program by getting user input of employee details:
 - a. employee ID: **C33**
 - b. name: **John Wong**
 - c. salary: **77777**
2. Remove employee with ID: **K66**
3. Update salary of employee with ID: **M88** to **88888**
4. Display all employees

[3]

Save your Jupyter Notebook as TASK1_<your class>_<your name>.ipynb..

2 Name your Jupyter Notebook as `TASK2_<your class>_<your name>.ipynb`.

A grocery store owner wants to use a database to store data for its inventory system. Currently the data are in three text files: "product.TXT", "supplier.TXT" and "transaction.TXT". The attributes in each file are described as follows:

product.TXT

- `ProductID`: A unique identifier assigned to each product.
- `ProductName`: The name of the product.
- `Category`: The classification of the product, for example, categories such as fruits, vegetables, or dairy.
- `UnitPrice`: The cost of one unit of the product.
- `StockQuantity`: The current quantity of the product available in the inventory.
- `SupplierID`: The identification code of the supplier who supplied this product.

supplier.TXT

- `SupplierID`: A unique identifier assigned to each supplier.
- `SupplierName`: The name of the supplier providing the product.
- `ContactNumber`: The phone number of the supplier.

transaction.TXT

- `TransactionID`: A unique identifier assigned to each transaction.
- `ProductID`: The identification code of the product involved in the transaction.
- `Quantity`: The amount of the product bought or sold during the transaction.
- `TransactionType`: Specifies whether the transaction pertains to a purchase or a sale.
- `TransactionDate`: The date on which the transaction took place.

Task 2.1

Write a Python program with the necessary SQL code and create a database `grocery.db` with three tables `products`, `suppliers`, and `transactions` to store the data from the corresponding data files. Define the primary and foreign keys, where necessary, for each table.

Save your Python program as `TASK2_<your class>_<your name>.ipynb` [4]

Task 2.2

In a new cell, write additional Python code to read in the data from each file and store each item of data in the corresponding tables within the `grocery.db` database.

Save and run your program.

[3]

Task 2.3

In a new cell, write additional Python code that accepts a user input for a **category** of a product and display the following in table form:

ProductID, ProductName, UnitPrice, SupplierID, and SupplierName.

Test your program by inputting 'Fruits' as the category.

Save your program. [6]

Task 2.4

The grocery store owner wants to display the transactions done on each product and display the results on a web browser.

Write a Python program and the necessary files to create a web application that

- receives a `TransactionType` and `TransactionDate` from a web page form,
- returns a web page that displays in table form: the `ProductName` and `Quantity`, whether purchased or sold on that date, in descending order of quantity.

Save your Python program as `TASK2_4_<your class>_<your name>.py` with any additional files/sub-folders as needed in a folder named `TASK2_4_<your class>_<your name>`.

Run the web application and input the `TransactionType` as 'Sale' and `TransactionDate` as '2024-05-03'.

Save your HTML output as `TASK2_4_<your class>_<your name>.html`. [12]

3 Name your Jupyter Notebook as: `TASK3_<your class>_<your name>.ipynb`.

Given a server program, your task is to create a client program for the Treasure Hunt game. The client program will connect to the provided server program and allow the player (client) to interact with the game by sending moves and receiving updates about the game state.

Game Overview:

The Treasure Hunt game takes place on a 5x5 grid map. Five treasures are randomly placed on the map. The player's objective is to find all five hidden treasures by navigating through the map – moving up, down, left, or right. The player finds the treasure by moving to a position where a treasure is hidden. The game ends when the player finds all the treasures.

Server Implementation (Provided in Python file `'TreasureHuntServer.py'`):

Server consists of a class called `'TreasureHuntServer'` which manages the game logic, communication with the client, and the game state.

1. Start Server and Player Connection
 - Server initialises a socket and binds it to an address and port, allowing it to listen for incoming connection from a client player.
 - Upon establishing a connection with a client player, the server prints a message to indicate that a player has connected.
2. Game Initialisation
 - Server initialises variables to manage the game state:
 - `found`: to keep track of the number of treasures found,
 - `player_position`: to store current position of the player on the map,
 - `map`: a 5x5 grid map represented by characters, where 'T' represents treasure, 'P' represents the player, and '.' represents an empty position
 - At the start of the game, server generates a random 5x5 grid map with five treasures and places the player in a random position on the map.
 - The initial state of the map, with treasure symbols hidden from the player, is then sent to the client.

<u>Sample map generated</u>	<u>Sample map sent to client (without 'T')</u>
.TT..	
...T.	
....P	P
.TT..	
.....	

3. Game Loop

- Server continuously listens for player moves received via socket connection.
- Upon receiving a move from the client player, server checks if the player has found a treasure and updates the player's position on the map accordingly.
- If a treasure is found, the server increments the `found` counter and informs the player by sending a message `"You found a treasure!"`.
- The server then checks if all five treasures have been found. If so, it ends the game and sends a message to the client player: `"Congratulations! You found all treasures."`. The socket connection is then closed.
- If the game has not ended, an updated map showing the player's current position is sent to the client player.

Client Implementation (Code this implementation):

The client program for the Treasure Hunt game facilitates player interaction with the game server and displays the game state to the player.

1. Client Player Connection

- Client initialises a socket and connects to the server using the provided address and port number.
- Upon connection to the server, it displays a welcome message along with game instructions:

```
Welcome to Treasure Hunt!
Use 'up', 'down', 'left', 'right' to navigate.
Find the treasures ('T') hidden in the map.
```

2. Game Initialisation

- Client receives the initial state of the map from the server and displays it.
- Sample map received from server:

```
.....
.....
.....P
.....
.....
```

3. Game Loop

- Client enters a game loop where it continuously prompts the player to enter their move using the following keys:
 - 'u' for moving up,
 - 'd' for moving down,
 - 'l' for moving left,
 - 'r' for moving right.
- Once the player enters a move, the client sends the move to the server.
- Client then waits to receive updates from the server regarding the game state.
- If the player finds a treasure, the client prints a message "You found a treasure!".
- If all treasures have been found, the client prints a message "Congratulations! You found all treasures." and exits the game loop.
- If the game is ongoing, the client continues to display the updated map state received from the server.

4. End of Game

- Upon exiting the game loop, the socket connection is closed.

Sample Run:

Server's side

```
Server is waiting for the player to connect...
Player connected at ('127.0.0.1', 52025)
Player is at (4, 2)
Current state of map (Server's view with Treasure symbols):
...TT
.....
.....
.....T
T.P.T
Sending map to player (without the Treasure symbols) ...
Player is now at (4, 3)
Current state of map (Server's view with Treasure symbols):
...TT
.....
.....
.....T
T..PT
Sending map to player (without the Treasure symbols) ...
Player is now at (4, 4)
Current state of map (Server's view with Treasure symbols):
...TT
.....
.....
.....T
T...P
```

Client's side

```
Welcome to Treasure Hunt!
Use 'up', 'down', 'left', 'right' to navigate.
Find the treasures ('T') hidden in the map.
.....
.....
.....
.....
..P..
Enter your move (up/down/left/right): r
.....
.....
.....
....P.
Enter your move (up/down/left/right): r
You found a treasure!
.....
.....
.....P
Enter your move (up/down/left/right):
```

Map as viewed on server's and client's side

By using the provided server program 'TreasureHuntServer.py', write the client program on the Jupyter Notebook platform.

Run the server and client programs and provide a sample output on the Jupyter Notebook.

Save your client program code as TASK3_<your class>_<your name>.ipynb

[14]